Assignment - 9.5

2303A51680

K Rishitha

Batch - 23

Problem 1:

String Utilities Function
Consider the following Python function:
def reverse_string(text):
return text[::-1]
Task:
1. Write documentation in:
o (a) Docstring
o (b) Inline comments
o (c) Google-style documentation
2. Compare the three documentation styles.
3. Recommend the most suitable style for a utility-based string library.

reverse_string.py > ...

```python
#Dockstring
def reverse_string(text):
    """
    This function a string as input and returns the reverse of that string
    """
    return text[::-1]
print(reverse_string.__doc__)
#google style documentation
def reverse_string(text):
    """
    Reverse the given string.
    Args:
    text(str):input string to be reversed
    returns:
    str:reversed string
    """
    return text[::-1]
print(reverse_string.__doc__)
#inline comments
def reverse_string(text):
    #use slicing with step-1 to reverse the string
    return text[::-1]
print(reverse_string.__doc__)  #this will print None since there is no docstring defined for the function
```

```
\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Desktop/AI-Lab/reverse_string.py

This function a string as input and returns the reverse of that string
\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Desktop/AI-Lab/reverse_string.py

This function a string as input and returns the reverse of that string


\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Desktop/AI-Lab/reverse_string.py

This function a string as input and returns the reverse of that string

\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Desktop/AI-Lab/reverse_string.py

This function a string as input and returns the reverse of that string

This function a string as input and returns the reverse of that string
```

```
Reverse the given string.
Reverse the given string.
Args:
text(str):input string to be reversed
returns:
str:reversed string

None
PS C:\Users\HP\Desktop\AI-Lab>
text(str):input string to be reversed
text(str):input string to be reversed
returns:
str:reversed string

None
PS C:\Users\HP\Desktop\AI-Lab>
```

Problem 2:

Password Strength Checker

Consider the function:

def check_strength(password):

return len(password) >= 8

Task:

1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

C: > Users > HP > Desktop > check_strength.py > ...

```python
#Docstring
def check_strength(password):
    """
    checks if password length is at least 8 characters
    """
    return len(password)>=8
print(check_strength.__doc__)

#inline comments
def check_strength(password):
    #passworsd must be at least 8 characters long
    return len(paassword)>=8
print(check_strength.__doc__)#this will print None since there is no docstring defined for the function

#google style documentation
def check_strength(passwors):
    """
    checks whether the password satisfies minimum length requirement
    Arges:
    password(str): password string
    Returns:
    bool:True if password length>=8,else False
    """
    return len(password)>=8
print(check_strength.__doc__)
```

```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Desktop/
check_strength.py

checks if password length is at least 8 characters

None
checks whether the password satisfies minimum length requirement
checks whether the password satisfies minimum length requirement
Arges:
checks whether the password satisfies minimum length requirement
Arges:
password(str): password string
Returns:
bool:True if password length>=8,else False
Returns:
bool:True if password length>=8,else False
bool:True if password length>=8,else False

PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

Problem 3:

Math Utilities Module

Task:

1. Create a module math_utils.py with functions:

o square(n)

o cube(n)

o factorial(n)

2. Generate docstrings automatically using AI tools.

3. Export documentation as an HTML file.

math_utils.py > ...

```python
def square(n):
    Returns the square of a number
    demonstrtes how to use docstring in python
    parameters:
    n (int): the nuber to be squared
    returns:int: Tghe square of n
    """
    return n*n
def cube(n):
    """
    Returns the cube of a numbers
    demonstrates how to use docstrings in python
    parameters:
    n (int): The number to be cubed
    Returns:int: The cube of n
    """
    return n*n*n
def factorial(n):
    """
    Returns the factorial of a number
    demonstrates how to use docstrings in python
    parameters:
    n (int): The number to calculate the factorial of
    Returns:int: The factorial of n
    """
    if n==0:   #check if n is 0 and return 1 if it is beacause factorial of 0 is 1
        return 1  #factoral of 0 is defined to be 1
    else:
        return n*factorial(n-1)   #Recursive call to calculate factorial of n
print(square.__doc__)
print(cube.__doc__)
print(factorial.__doc__)
```

```
PS C:\Users\HP\Desktop\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Desktop/AI-Lab/math_utils.py

Returns the square of a number
demonstrtes how to use docstring in python
parameters:
n (int): the nuber to be squared
returns:int: Tghe square of n


Returns the cube of a numbers
demonstrates how to use docstrings in python
parameters:
n (int): The number to be cubed
Returns:int: The cube of n


Returns the factorial of a number
demonstrates how to use docstrings in python
parameters:
al of
Returns:int: The factorial of n
```

Problem 4:

Attendance Management Module

Task:

1. Create a module attendance.py with functions:

o mark_present(student)

o mark_absent(student)

o get_attendance(student)

2. Add proper docstrings.

3. Generate and view documentation in terminal and browse

attendance.py > ...

```python
attendance={}
def mark_present(students):
    """
    Marks a student as present in the attendance record
    Parameters:
    student(str): the name of the student to be marked as present
    """
    attendance[student]="Present"
def mark_absent(student):
    """
    Marks a student as absent in the attendance record
    Parameters:
    student(str): The name of the student to be marked as absent
    """
    attendance[student]="Absent"
def get_attendance(student):
    """
    Return the attendance status of a student
    PArameters:
    student(str):The name of the student whise attendance is to be retrieved
    Returns:
    str:The attendance status of the student
    """
    return attendance.get(student,"Not found")
import attendance
help(attendance)
```

```
PS C:\Users\HP\Desktop\AI-Lab> python -m pydoc -w attendance
Help on module attendance:

NAME
    attendance

FUNCTIONS
    get_attendance(student)
        Return the attendance status of a student
        PArameters:
        student(str):The name of the student whise attendance is to be retrieved
        Returns:
        str:The attendance status of the student

    mark_absent(student)
        Marks a student as absent in the attendance record
-- More  --
```

Functions

get_attendance(student)

```
Returns the attendance status of a student.
  Parameters:
  student (str): The name of the student whose attendance is to be retrieved.
  Returns:
  str: The attendance status of the student.
```

mark_absent(student)

```
Marks a student as absent in the attendance record.
  Parameters:
  student (str): The name of the student to be marked as absent.
```

mark_present(student)

```
Marks a student as present in the attendance record.
Parameters:
student (str): The name of the student to be marked as present.
```

Data

attendance = {}

Problem 5:

File Handling Function

Consider the function:

```
def read_file(filename):
with open(filename, 'r') as f:
return f.read()
```

Task:

1. Write documentation using all three formats.
2. Identify which style best explains exception handling.
3. Justify your recommendation.

read_file.py > read_file

```python
#Docstring style:
def read_file(course):
    """
    Reads the content of a file and returns it as a string
    Parameters:
    Returns:
    str:The content of the file
    Raises:
    FileNotFoundError:If the specified file does not exist
    IOError:If an I/O error occurse while reading the file
    """
    try:
        with open(course,'r')as f:
            return f.read()
    except FileNotFoundError:
        print(f"Error:The file'{course}'was not found.")
    except IOError as e:
        print(f"An I/O error occured:{e}")
        raise
#google style:
def read_file(filename):
    """
    Reads the cintent of a file returns it as a string
    Args:
    filename(str): the name of the file to be read
    Returns:
    str:The content of the file
    Raises:
    FileNotFoundError:If the specified file does not exist
    IOError:If an I/O error sccors while reading the file
    """
```

```python
    FileNotFoundError:If the specified file does not exist
    IOError:If an I/O error sccors while reading the file
    """
    try:
        with open(filename,'r')as f:
            return f.read()
    except FileNotFoundError:
        print(f"Error:The file'{filename}'was not found.")
        raise
    except IOError as e:
        print(f"An I/O error occured:{e}")
        raise
#inline comments
def read_file(filename):
    #open the file in read mode
    #if the file does not exist, python raises FileNotFooundError
    with open(filename,'r')as f:
        #read the entire contents of the file
        return f.read()
```

```
PS C:\Users\HP\Desktop\AI-Lab> python -m pydoc -w course
{'name': 'Introduction to Computer Science', 'credits': 4}
Course with ID HIST101 not found in the catalog.
wrote course.html
PS C:\Users\HP\Desktop\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Desktop/A
I-Lab/course.py
{'name': 'Introduction to Computer Science', 'credits': 4}
Course with ID HIST101 not found in the catalog.

This function takes a course ID as input and simulates retrieving course information from a course catalog.
Args:course_id (str): The unique identifier for the course to be retrieved.
Returns:
dict: A dictionary containing the course information if found, or a message indicating that the course was not found.

None
{'name': 'Introduction to Computer Science', 'credits': 4}
Course with ID HIST101 not found in the catalog.

This function takes a course ID as input and simulates retrieving course information from a course catalog.
Args:course_id (str): The unique identifier for the course to be retrieved.
Returns:
dict: A dictionary containing the course information if found, or a message indicating that the course was not found.

None

This function takes a course ID as input and simulates retrieving course information from a course catalog.
Args:course_id (str): The unique identifier for the course to be retrieved.
Returns:
dict: A dictionary containing the course information if found, or a message indicating that the course was not found.

None
```